

on the use of certain features in the devices, and industry environments that use these features.

Let us begin with getting an insight into the salient features of the CELF specifications [1]. Due to the volume of information that needs to be presented, we have split the article into two parts. In Part I we will discuss boot-up time reduction and power management. We will focus on real-time characteristics and size reduction in Part II.

Boot-up time reduction

The need for a reduction in the boot-up time is obvious. People want to get started fast. This is more so if you are talking about a handheld device or a television—users expect them to work as soon as they are turned on.

Tim Bird, chair of the Architecture Group in CELF [2], said that there are four principles of reducing boot-up time. Whatever you do during boot-up:

- Do it fast (see if you can speed up an activity during boot-up)
- Do it in parallel (get things done during boot-up, in parallel)
- Do it later (postpone unnecessary activities that are traditionally taken care of during boot-up)
- Don't do it at all (see if you can avoid doing an activity during boot-up)

There are multiple ways in which you can reduce boot-up time.

Let us explore each of the following techniques in greater detail:

- XIP
- IDE no probe
- Disabling console output
- Delay loading drivers
- Smaller kernel
- Excellent quick boot

XIP

XIP stands for eXecute In Place, which means executing the kernel in the same place it is stored (i.e., from the ROM, persistent memory storage).

XIP comes under the 'Do it Fast' category—while performing an activity during boot-up, find ways to finish it quicker. In the conventional method, the kernel image is copied from ROM to RAM, uncompressed in a different location in the RAM and then executed. (See Figure 1.)

But XIP is a method for executing code directly from ROM or Flash, without first loading it into RAM. Kernel XIP refers to the capability for a Linux kernel to be executed directly from a persistent, read-only memory or media type (Figure 2). When the kernel is executed in this fashion, the boot loader can avoid loading the kernel from persistent storage on to RAM, uncompressing it, and saving it into the system RAM [3], [4] and [5].

This will impact start-up time, RAM and ROM footprint, and execution performance. Reduction in boot-up time occurs mainly in two phases. The time it takes to copy the kernel image from the ROM to the RAM is reduced, in addition to a reduction in the time it takes to uncompress the image after copying it on to RAM. For a slow processor, this can take around 1.5 to three seconds.

Kernel XIP has the following advantages, with some overhead of running the kernel due to frequent access of

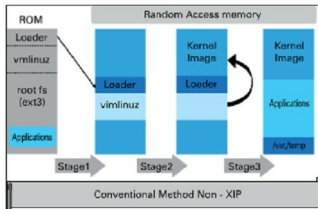


Figure 1: Different phases in conventional boot loading procedure

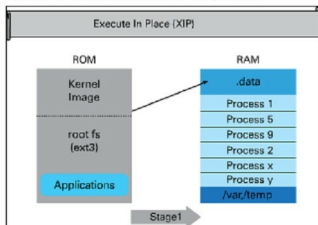


Figure 2: Different phases involved in XIP procedure

Flash memory:

- It provides faster boot-up time by reducing the time to copy from ROM to RAM and by completely eliminating decompress time.
- It also has smaller RAM footprint since the kernel text portion is not loaded in to RAM.

IDE no probe

Linux probes for the presence of different sectors of the hard drive, the USB, SDRAM drive and other such drives. This time could be saved if we disable the probing of the unused drive, by using *ide no probe = <x>* (where <x> is the drive name) [6] and [7]. Data provided by Tim Bird says that the time to probe an empty IDE (Integrated Drive Electronics) device is approximately 1.3 sec [6].

'IDE no probe' comes under the category of 'Do it Later'—see whether you can postpone an activity done during the boot-up [6], [7] and [8].

Disabling the console output

The console output can be disabled for faster boot-up time.

In actual devices or in production times, the console output is not needed [9]. Console output can be disabled with the *quiet* argument in the Linux kernel command line (boot loader settings):

```
root@dev/rw00 ~# init = /startup.sh quiet
```